INF2006 Cloud Computing & Big Data
Team Project 1

CafeStock: stock, forecasting and rosters for small cafés

Group: P1-3

Hu Shi Yao (2500892), Mohamed Raihan Bin Ismail (2503274), Jason Woo (2502583), Goh Jason Fadil (2502643) , Ho Shang Jay (2500526)

10th October 2026

1. Problem, users and success criteria

1.1 Problem statement

CafeStock is a cloud-hosted SaaS web app for independent cafés that replaces scattered verbal, paper and chat-based record-keeping with one reliable system: every delivery, usage, write-off, correction and stock count goes into an append-only ledger so each ingredient's stock level is explained by its history, a nightly forecasting job predicts the next week's demand per ingredient and produces reorder recommendations (what, how much and why) to prevent stock-outs, spoilage and duplicate or missed orders, and the same app holds the weekly roster so the owner can plan shifts and each staff member can see their own.

1.2 Users

CafeStock has two roles, each an Amazon Cognito user group. Small cafés rarely have IT staff, so the owner runs the system while an administrator (a team member acting as IT support) creates, removes and re-roles accounts in the Cognito console; there is no self-sign-up.

Owner/manager. Accountable for stock and ordering. Needs a current view of every ingredient, the history behind each number, and a clear recommendation of what to reorder. Only managers can open the Forecast page and its API, which show demand forecasts, reorder recommendations and how accurate the forecasting model has been. Managers also plan the roster and see everyone's hours.

Staff (group staff). Front-line workers who receive deliveries, use ingredients, throw away spoiled stock and count stock at closing. They record these movements, check current stock and keep the item and supplier lists up to date, and see their own upcoming shifts under "My shifts". They cannot see the forecast, colleagues' shifts or the team list, and the API refuses such requests (HTTP 403).

1.3 Success criteria

SC-1. Every stock change goes through the ledger. An item's stock level changes only inside the same database transaction that appends a movement row recording the change, who made it and the resulting level. The app never edits or deletes movement rows. *Measure:* after a delivery of 1,000 ml of milk and a usage of 220 ml, stock changes by exactly +780 ml and two ledger rows show the running level.

SC-2. Staff can record every kind of stock movement. Delivery, usage, waste, adjustment and stock count each take one form submission, saved with the signed-in user and a timestamp. *Measure:* each movement type appears in the Activity log with the staff member's name.

SC-3. Invalid entries are rejected. *Measure:* 100% of a defined set of invalid inputs is rejected with a clear message and writes no ledger row: a zero or negative quantity, a zero adjustment, more than three decimal places, an unknown or archived item, an unknown movement type, a removal larger than the stock on hand, and a date in the future or more than 90 days ago.

SC-4. Corrections preserve history. Stock is never edited in place. *Measure:* after an adjustment or a stock count, the original entries and the correction all remain in the ledger, each showing who recorded it.

SC-5. Owners know what to reorder before stock runs out. The Forecast page lists each ingredient's 7-day forecast, stock on hand, reorder point, recommended order quantity and status (OK, REORDER or URGENT), urgent items first, with the calculation explained. [1] *Measures:* (a) the forecast beats a naive "same day last week" forecast in a walk-forward backtest (MASE below 1.0); (b) in a full-year replay, forecast-driven ordering meets a higher share of demand from stock than ordering to a fixed level; (c) a nightly run that finds urgent items e-mails the owner once through Amazon SNS.

SC-6. Access matches roles and cafés. *Measure:* requests without a valid Cognito token are rejected (401), including forged and expired tokens; staff are refused the forecast (403); a user never sees or changes another café's data.

SC-7. Stock stays correct under concurrent use, and the service survives a server failure. *Measure:* simultaneous movements for one item leave stock equal to the net of all accepted movements, never negative; terminating one of two app servers causes at most a few seconds of failed requests, and the Auto Scaling group replaces it automatically.

SC-8. One place for the roster, with nobody double-booked. *Measure:* a shift the manager creates appears in the assigned employee's My shifts; staff see no one else's shifts; a shift that overlaps one the person already has is refused (409), including when several are sent at the same moment.

2. Solution overview and architecture

2.1 Overview

CafeStock is a multi-tenant web application (software as a service): each café is an organisation, and every record belongs to exactly one. **Stock** is where staff record deliveries, usage, waste, adjustments and stock counts, and see the Activity log and each item's history. **Items** and **Suppliers** hold the ingredient list (unit, cost, supplier, reorder point, spoilage cap) and supplier contacts. **Forecast** (managers only) shows, for each ingredient, next week's forecast demand, stock on hand, reorder point, recommended order and status, with the calculation explained. **Roster** (managers)is the week's shifts by day, with hours per person and the team list; staff see it as **My shifts**, their own shifts only.

Every night an AWS Lambda function forecasts each ingredient's demand, compares a LightGBM model with a moving-average baseline, keeps whichever has been more accurate recently, and writes forecasts and reorder recommendations to the database [2]. The demo café is loaded with a year of a real coffee shop's sales, expanded into eight ingredients (section 5.1).

2.2 Architecture

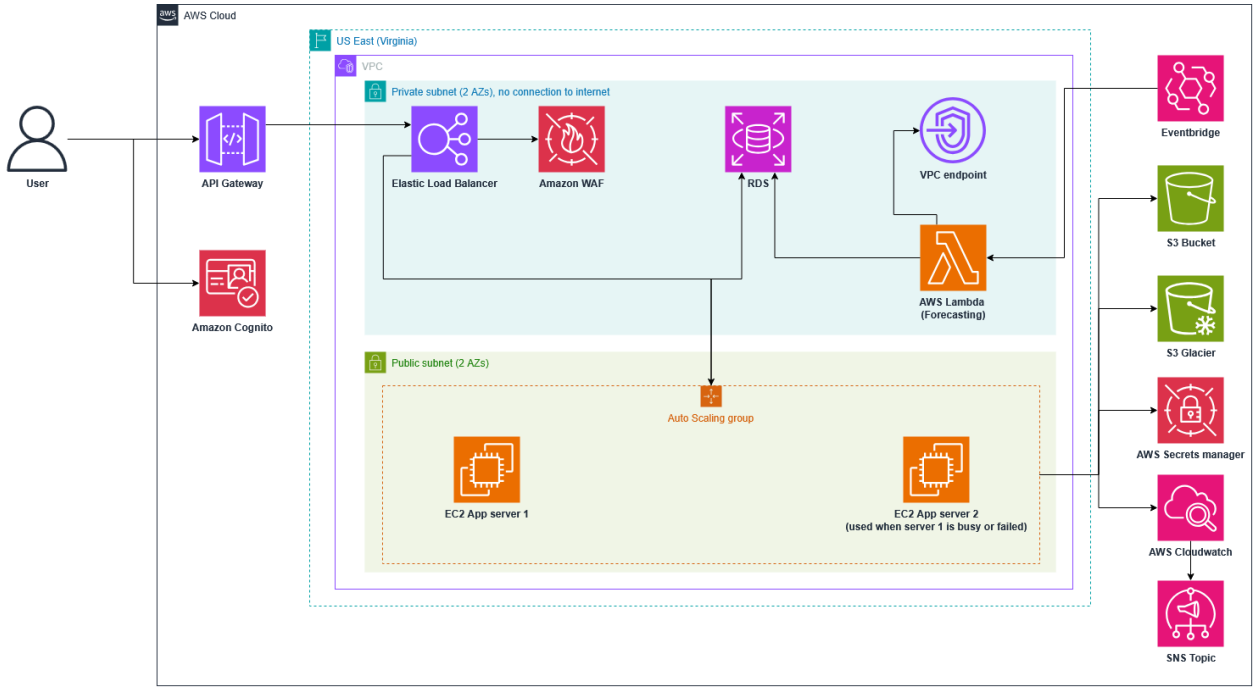


Figure 1. CafeStock on AWS (us-east-1, two Availability Zones); evidence/architecture.png

- **Entry and web tier:** the browser reaches an **API Gateway HTTP API** over HTTPS. A VPC link forwards requests to an **internal Application Load Balancer** protected by **AWS WAF**, which spreads them across an **EC2 Auto Scaling group** (1–2 t3.micro servers in two Availability Zones). Each server runs the Node.js API, which also serves the React web app. Users sign in with **Amazon Cognito**.
- **Data: Amazon RDS for PostgreSQL 17** in private subnets, encrypted at rest, accepting only TLS connections. **Secrets Manager** generates and holds the database password.
- **Forecasting and operations:** an **EventBridge** rule starts the **Lambda** function (Python 3.13) at 02:00 Singapore time, loading the model from **S3**. **CloudWatch** holds logs, metrics, dashboards and alarms; **SNS** e-mails the team.

Service model: IaaS for the web tier (we configure the EC2 servers), PaaS and serverless for data and background work (RDS, Lambda, API Gateway, S3, Secrets Manager) and managed identity (Cognito); to its users, CafeStock is SaaS. **Deployment model:** public cloud, one region (us-east-1) across two Availability Zones, in an AWS Academy Learner Lab account. The whole

system is defined as code in six CloudFormation templates (src/infra/): Cognito, artifacts bucket, forecast, network, database and app.

3. Cloud service/deployment choices and trade-offs

3.1 Assumptions, constraints and expected workload

- **Users:** each café has 1–2 managers and 4–10 staff; an administrator creates accounts.
- **Recording:** staff enter usage by hand (for example at the end of a shift); there is no link to the till.
- **Catalogue**: 8–30 tracked ingredients per café, from suppliers with 1–3 day lead times.
- **Traffic:** under 1 request(s) per café on average. We design for peaks of 2 requests(s) per café at opening and closing. One deployment is sized for 20 cafés in one time zone, so their peaks coincide: about 40 requests(s) at worst. That is within API Gateway's 50 requests(s) limit, and short bursts up to 100 are absorbed.
- **Data volume:** 20 cafés × 80 movements a day ≈ 0.6 million ledger rows a year, small for a db.t3.micro with 20 GB.
- **Availability:** needed during opening hours (about 7 am–10 pm). A few minutes' outage is tolerable: staff note movements on paper and enter them later, since entries can be backdated up to 90 days (SC3).
- **Constraints:** AWS Academy Learner Lab ($50 credit, no IAM role creation, no CloudFront, EC2 stopped when a session ends); public or synthetic data only; four students, six weeks.
- **Cost control:** paused between sessions, no NAT gateway, single-AZ database, Lambda for the nightly job (section 7.1).

The complete choices and trade offs are given in Appendix I (Table 1)

4. Implementation, data design and security controls

4.1 Implementation

The **API** is Node.js 24 with Express 5, Prisma 7 and zod validation; the **web app** is React 19 (Vite, TanStack Query), the **forecast engine** is Python (LightGBM 4.7, pandas), shared by the Lambda and the analytics notebook so both use identical code.

Recording stock. All stock changes go through one service function. Inside a single transaction it locks the item's row (SELECT … FOR UPDATE), computes the new level, refuses it if it would go below zero (HTTP 409), updates the running total, appends the movement row and adds any usage to that day's demand history (in the café's own time zone). Two people recording at the same moment therefore queue instead of overwriting each other. **Booking shifts** use the same pattern: the person's row is locked, then any overlapping shift of theirs refuses the booking (HTTP 409). Shifts are entered in café time and stored as exact instants.

Deployment. npm run app:build produces a versioned release file that is uploaded to S3. At launch, each EC2 server runs deploy/ec2/bootstrap.sh: it installs Node.js 24 (checked against the published SHA-256), downloads the release, reads the database secret, applies database migrations, imports the demo data if the database is empty, and starts the app as an unprivileged systemd service with the CloudWatch agent. Servers keep no state, so any of them can be replaced at any time.

Nightly forecast. The Lambda function loads the model from S3, connects to RDS over verified TLS, takes a database lock so two runs never overlap, scores both methods, writes forecasts and recommendations in one transaction, and publishes metrics to CloudWatch.

4.2 Data design

PostgreSQL holds eleven tables (src/backend/prisma/schema.prisma):

Group	Tables	Design notes
Tenancy and people	Organizations, User	Every other row carries an org_id. A user is linked to their Cognito account; the café is taken from that account, never from the request.
Catalogue	Supplier, Item	Quantities are exact decimals, since stock is in ml and g. Items keep a running on_hand, a reorder point and a spoilage cap. Archiving hides an item but keeps its history.
Ledger and demand	InventoryMovement, DemandHistory, DatasetImport	Append-only movements with signed change, level after, user and time. Demand is one row per item per day, from the dataset or from recorded usage. Each import records its files, row counts and checksums.
Forecasting	ForecastRun, Forecasts, ReorderRecommendation	Each run records the model used, its accuracy against the baseline and the model file's location, so every recommendation can be traced to the model that made it.
Scheduling	Shift	One row per person per shift, or an open shift with no person. A CHECK keeps each shift 15 min–16 h; people who leave keep their past shifts.

Composite foreign keys (id, org_id) make it impossible, in the database itself, for one café's item to reference another café's supplier or movement. Schema changes are versioned Prisma migrations, applied the same way locally and on AWS.

4.3 Security controls

The full threat-to-control map, with the file implementing each control and the test that shows it, is in evidence/threat-control-map.md (threats T1–T15). The main controls:

Threat	Control
Broken access control; one café seeing another's data	Role read from the verified token's Cognito groups on every request; forecast, team and roster changes require manager; staff shift queries are limited by the server to their own id; every query scoped by org_id, plus composite foreign keys.
Forged or expired tokens; password attacks	Cognito signs tokens; the API checks signature, expiry, issuer, token type and client ID. Cognito handles passwords and lockout; 10-character policy; no self sign-up.
Injection and cross-site scripting	Parameterised queries (Prisma); zod validation of every input; 100 kB body limit; React escaping; strict Content-Security-Policy; WAF managed rules.
Eavesdropping	HTTPS at API Gateway; TLS with certificate verification (verify-full) from servers and Lambda to RDS; RDS refuses unencrypted connections.
Exposed database or servers	RDS in private subnets, not publicly accessible, port 5432 open only to the app and Lambda security groups; internal load balancer that answers 403 without API Gateway's secret header; no SSH port.
Leaked secrets; compromised server; floods	Password generated by Secrets Manager and stored in a root-only file; no access keys anywhere; IMDSv2 only; app runs as an unprivileged user with systemd hardening; API Gateway throttling; RDS backups.

Known gaps. The Learner Lab does not allow creating IAM roles, so servers and Lambda use the shared LabRole, which has more permissions than they need; a real account would give each a least-privilege role. The app also connects as the database administrator, the password is not rotated, and API Gateway reaches the internal load balancer over HTTP inside the VPC (a certificate needs a custom domain).

5. Analytics or AI/ML feature: data, method, evaluation and limitations

5.1 Data

[3] The source is the Kaggle *Coffee Sales Dataset*: 3,547 point-of-sale transactions from one shop, 1 March 2024 to 23 March 2025, with no personal data [4] analytics/build_ledger.py expands each drink sold through a recipe table (a latte: 18 g of beans, 220 ml of milk, a cup, a lid, a napkin) into daily demand for eight ingredients (8 × 388 days = 3,104 rows). Deliveries, write-offs and counts are simulated for a realistic ledger; only observed demand is used to evaluate forecasts. The model is trained on a frozen, checksummed snapshot of the app's data.

5.2 Method

Forecasts. Two candidates predict each ingredient's daily demand seven days ahead: a **14-day moving average** (the baseline) and **LightGBM** lgbm-v1, one global gradient-boosted model for all ingredients. Its ten inputs describe recent demand, scaled by the item's 28-day average so millilitres and pieces look alike, and the weekly rhythm (weekday, same weekday one and two weeks back, 7- and 14-day averages, spread, share of zero days). Settings were fixed in advance, not tuned on the test weeks.

Choosing each night. Each run re-scores both candidates on the last eight weeks, forecasting each week only from the days before it, and uses whichever has the lower MASE. Items with under 28 days of history always use the baseline.

From forecast to recommendation. The system looks at how much of each ingredient you use per day and how unpredictable that usage is. It then calculates a minimum stock level (the reorder point) that covers the days you'd be waiting for a delivery to arrive, plus a buffer for days when you use more than average. That buffer is sized so you have roughly a 95% chance of not running out while waiting. When stock drops below that level, the item is flagged as REORDER. The suggested order is enough to cover about a week of demand on top of the buffer, but never more than the item can sell through before it expires. If stock will not last until a delivery could arrive, it is URGENT.

5.3 Evaluation

Accuracy. Walk-forward over 47 weeks × 8 items: before each week the model is retrained only on data known by then, then scored on the same 2,632 days as the baseline and a naive reference ("same day last week"). MASE is error ÷ the reference's error, so below 1.0 beats it.

Method	MASE ↓	Weekly WAPE ↓	WAPE
Same day last week (reference)	1.000	22.4%	56.7%
14-day moving average (baseline)	**0.827**	23.4%	46.4%
LightGBM lgbm-v1	0.855	**21.4%**	47.9%

Both methods meet SC5(a). Day by day the moving average is more accurate on all eight items; on weekly totals, which drive ordering, LightGBM is slightly better (closer in 192 of 376 item-weeks), mainly for fresh milk. **The ML model does not clearly beat a well-chosen baseline on this data**, which is why the app re-scores both every night instead of assuming the model wins. In the most recent eight weeks LightGBM scores 1.088 against the baseline's 1.126, so the current nightly run uses lgbm-v1.

Business value. analytics/evaluate_policy.py replays the whole year day by day under different ordering policies. Ordering to a fixed level (no forecast) meets 91.7% of demand from stock; every forecast-driven policy reaches 95.4 -- 95.9%, about 100 fewer stock-out days a year, with waste below 1% of demand (SC5b met). Ordering milk twice a week instead of weekly lifts milk from 82.5% to 91.4%, a bigger effect than the choice of forecasting method.

Reproducibility. 19 automated tests check that all methods are scored on identical days, that changing future data changes no earlier forecast (no leakage), that training is deterministic and that the reported metrics reproduce. Record: evidence/test-data-ai.md.

5.4 Limitations and responsible use

- **One shop, one year:** patterns may not carry over to other cafés, and one year gives only one example of each season.
- **Assumed recipes, simulated deliveries:** real portions vary by barista and cup size.
- **Sparse items:** cocoa powder has no demand on 64% of days, which limits any daily forecast.
- **No outside information:** holidays, weather and promotions are not modelled; they are the most likely way to improve the model as the café's own data grows

6. Testing, scalability/resilience and monitoring results

6.1 Test plan and results

Automated tests run offline with npm run evidence:tests, 277 backend API tests against a PostgreSQL database, 123 web-app tests, 85 forecast-engine and Lambda tests, and 19 analytics tests.

The four required tests are presented below, with a test plan, expected results and actual results for each test. The evidence records contain the detailed setup, steps, dates and artefacts.

6.1.1 Test 1 - Functional

Test plan: On the deployed AWS application, sign in as staff, record a delivery and a stock count, and check the stock view and Activity log. Sign in as a manager to check the forecast, assign a shift and attempt an overlapping booking. Sign in as the staff member to check My shifts.

Expected results: The recorded delivery and count should be reflected in stock, the Activity log and the manager's forecast. The assigned shift should appear in the staff member's My shifts, and an overlapping booking should be refused.

Actual results: Passed on 2026-10-07. The functional workflow reflected the 2,000 ml Fresh milk delivery in the Activity log and forecast. The roster prevented an overlapping shift and displayed the assigned shift to the staff user. The AWS smoke test also passed 10/10 checks, including application and database health and successful loading of the web application.

6.1.2 Test 2 - Security

Test plan: Use a staff account to attempt access to the manager-only forecast and colleagues' shifts. Send anonymous and forged-token API requests, submit a script probe to AWS WAF, and check that the RDS database and internal load balancer are not directly reachable from the internet.

Expected results: Staff should be refused access to restricted functions and colleagues' shifts. Anonymous and forged-token requests should return HTTP 401. AWS WAF should block the script probe with HTTP 403, and the database and load balancer should not be directly exposed to the internet.

Actual results: Passed on 2026-10-07. The reported security test confirmed the access controls. The AWS smoke test passed 10/10 checks: anonymous access returned 401 AUTH_REQUIRED, the forged manager token returned 401 INVALID_TOKEN, and the WAF script probe returned 403. Cognito authentication, HTTPS, Content-Security-Policy and X-Content-Type-Options checks also passed. Infrastructure evidence showed an internal load balancer and restricted database access.

6.1.3 Test 3 - Data and AI

Test plan: Run the analytics checks to verify that the forecast model and baseline are evaluated fairly and reproducibly without using future data. Compare their MASE values and review the selected model in successful forecast-run logs.

Expected results: All analytics checks should pass. Model and baseline evaluation should use a fair comparison with no future-data leakage, and the resulting metrics and model selection should be reproducible.

Actual results: Passed: 19/19 checks on 2026-10-01. The analytics checks validated the evaluation process. The recent successful forecast runs reviewed through Logs Insights selected lgbm-v1, with model MASE **1.0878** compared with baseline MASE **1.1261**. These operational monitoring results are described further in Section 6.3.

6.1.4 Test 4 - Resilience

Test plan: Begin with two application servers and send continuous requests during a 15-minute availability watch. Terminate one server, observe Auto Scaling replacement and load-balancer health, and measure successful requests, failed requests and the longest outage (SC7).

Expected results: The application should maintain near-continuous availability, with no failed requests or only a few seconds of interruption. Auto Scaling should automatically replace the terminated server and restore healthy application capacity.

Actual results: On **2026-10-07**, the watch sent **882 requests**: **828 returned HTTP 200** and **54 failed**, giving **93.88% availability**. The longest outage was **46 seconds**, and the slowest response was **2319 ms**. Failed requests began at approximately **12:17:44 UTC**. Auto Scaling launched a replacement, and the target group returned to two healthy targets across us-east-1a and us-east-1b; the exact replacement InService time was not recorded. **Self-healing worked, but the availability target was not fully met** because CPU target-tracking scaled desired capacity from **2 to 1** during recovery. Section 6.2 explains the proposed improvement.

Automated suites run offline (npm run evidence:test) : 277 backend, 123 web-app, 85 forecast-engine and 19 analytics tests (evidence/test-runs/SUMMARY.md). Table 2 shows the four required tests, run on the AWS deployment. See table 2 for the complete technical analysis

6.2 Scalability and resilience

The web tier is stateless, so it scales and heals by adding or replacing servers. The Auto Scaling group keeps 1–2 servers across two Availability Zones and adds one when average CPU exceeds **60%**. The load balancer checks /health every 15 s and the group replaces any server it marks unhealthy. Updates roll out one server at a time, and API Gateway throttles floods before they reach the servers. Correctness under concurrent use is tested directly: 12 simultaneous “used 1 L” requests against 10 L of stock produce exactly **10 successes and 2 refusals**, and stock ends at zero (SC7); six simultaneous bookings of one person produce exactly **one shift** (SC8).

Test 4 result: the 15-minute availability watch sent **882 requests**, of which **828 returned HTTP 200** and **54 failed**. Measured availability was **93.88%**, with a **46-second longest outage** and a slowest response of **2319 ms**. Failed requests began at approximately **2026-10-07 12:17:44 UTC**. The Auto Scaling group detected the terminated/unhealthy EC2 instance and automatically launched a replacement, after which the target group returned to two healthy targets across us-east-1a and us-east-1b. The exact replacement InService time was not recorded.

The self-healing mechanism worked, but the availability target was **not fully met**. The result was worse than the expected near-continuous availability because the CPU target-tracking scaling policy simultaneously reduced desired capacity from two servers back to one while the recovery test was in progress. For a future test, minimum and desired capacity should temporarily be fixed at two, or scale-in should be suspended, so that recovery and scale-in do not occur at the same time.

Limits: the single-AZ database is the main remaining single point of failure; next steps are a Multi-AZ database and a larger instance class.

6.3 Monitoring results

Two CloudWatch dashboards show requests, p95 response time, 5xx errors, healthy servers, CPU and WAF blocks, and each night’s items to order, urgent items, both models’ MASE and run time. Seven alarms e-mail the team through SNS: 5xx errors, unhealthy servers, forecast failed, forecast missed, urgent items, database CPU and low storage. Server and Lambda logs are kept for **14 days**.

Operational test: a Lambda run with {"fail": true} was deliberately triggered at **2026-09-30 04:15:37 UTC**. The cafestock-forecast-failed alarm changed from OK to ALARM at **04:16:05 UTC**, approximately **28 seconds** later, and returned to OK at **04:31:05 UTC**. The exact SNS e-mail receipt time was not recorded, but the alarm state transition and SNS action were visible in CloudWatch alarm history.

Logs Insights: recent successful forecast runs selected lgbm-v1, with a model MASE of **1.0878** compared with **1.1261** for the 14-day moving-average baseline. Successful runs followed the sequence job.started → model.ready → cafe.published → job.finished, publishing forecasts for one café with zero skipped and zero failed cafés. The CloudWatch forecast dashboard showed a run time of approximately **10.7 seconds**. The only failure observed in the queried period was the deliberate job.test_failure used for the monitoring test.

7. Cost, sustainability and operational considerations

7.1 Cost

Approximate on-demand prices in us-east-1 while everything is running (src/docs/aws-deploy.md):

Component	Per day
Internal Application Load Balancer	$0.54
RDS db.t3.micro + 20 GB storage	$0.51
One t3.micro app server (with public IPv4 and 8 GB disk)	$0.39
Secrets Manager secret + VPC endpoint	$0.25
WAF web ACL and rule groups	$0.23
API Gateway, Lambda, CloudWatch, SNS	a few cents
Total while running	**≈ $1.90 (≈ $57 a month)**

Between sessions we pause it (about $0.35–1.10 a day). Observed Learner Lab spend: **$1.90**. Cost drove no NAT gateway (about $1 a day), a single-AZ database and Lambda for the forecast; being multi-tenant, one deployment can serve many cafés, so the cost per café falls as it grows.

7.2 Sustainability

Resources are right-sized and switched off when idle: burstable t3.micro instances that scale on demand, a forecast that runs a few minutes a night instead of on an always-on server, 14-day log retention and 30-day expiry of old S3 versions; ARM-based Graviton instances would be the next step. The larger effect is in the café: better-timed orders reduce stock-outs and wasted perishable food.

7.3 Operational considerations

- **Deploying and rolling back:** a new release file in S3 plus a stack update; servers are replaced one at a time, and S3 keeps previous releases for 30 days.
- **Backups and runbooks:** RDS daily backups and a final snapshot on deletion; step-by-step deployment, pausing and recovery in src/docs/aws-deploy.md.
- **Learner Lab behaviour:** when a session ends, AWS stops EC2 but not RDS, so the database is stopped by hand (the nightly forecast then raises its failure alarm, as expected).
- **Before real use:** least-privilege IAM roles, a restricted database user, automatic password rotation, a Multi-AZ database and a custom domain with its own certificate

8. Team contribution, ethical considerations and reflection

8.1 Team contribution

Member (student ID)	Role	Main contributions and evidence owned
Mohamed Raihan Bin Ismail (2503274)	Team Leader, Web Developer (Full stack)	● Create the database and the web application locally. ● Implemented Prisma schema with 14 models and enums ● Set up Backend API CRUD for suppliers, items and stock movements. ● Created manager-only versus staff-only access. ● Implemented error handling and security headers, helmet, CORS and JSON body limit. ● Set up UI/UX for the website. ● Implemented bash scripts for the user to run locally ● Implemented Docker image and containers.
Ho Shang Jay (2500526)	Web Developer, AWS Developer	● Helped with implementing the PostgreSQL database (Prisma schema and migrations) and React web to ran it locally in Docker. ● Deployed the system in AWS Academy Learner Lab using six CloudFormation stacks: Cognito, S3, Forecast Lambda, VPC, RDS, and the app stack. ● Connected the web app and API to RDS PostgreSQL, S3 and Cognito. ● Implemented Cognito sign-in with staff and manager roles, with role checks enforced on the server. ● Improved the forecasting model: LightGBM (lgbm-v1) compared against a 14-day moving-average baseline.
Hu Shi Yao (2500892)	Data Analyst	● Transformed the original coffee sales dataset, which lacked the depth of data for our inventory analysis as-is , into five usable datasets: an ingredient list, a recipe table mapping each drink to its ingredients, a 17,049-row stock-movement ledger, and daily and weekly demand tables for eight ingredients. ● Developed the base forecast model and developed reorder-point method: a moving average evaluated by walk-forward backtest against a naive baseline using MASE. This became the baseline the LightGBM model was compared against. ● Wrote the data/AI test record and the analytics evaluation.
Goh Jason Fadil (2502643)	Demo Video Lead, Product Requirements & Ideation	● Worked with the team to shape the product idea: stock, forecasting and roster app for small cafes as well as the product requirements, problem statement and success criterias (SC-1 to SC-8) ● Defined the 3 core workflows (WF1-WF3). Led the demo video: liaised the AWS deployment, recorded the screen captures and directed the edits.

Member (student ID)	Role	Main contributions and evidence owned
Jason Woo (2502583)	AWS deployment and operations lead	● Performed the deployed functional, security, resilience and CloudWatch monitoring tests, and collected the AWS evidence and screenshots used in the final report. ● Hosted and operated the team's main CafeStock deployment in my AWS Academy Learner Lab using the deployment procedure prepared by the team. Managed application updates and redeployments through S3 and CloudFormation. ● Helped formulate the AWS cost estimates and cost-control decisions. Also contributed to the testing, monitoring, resilience and cost sections of the report.

Details and each member's reflection: TEAM_CONTRIBUTIONS.md. The team used an AI assistant during development; its use and how output was checked are declared in AI_USE_DECLARATION.md.

8.2 Ethical considerations

- **Data:** a public dataset with no personal information, used under its licence and credited in data/README.md. Simulated parts of the data are labelled as simulated.
- **Staff privacy and fair hours:** the ledger records who made each change for accountability, and should not be used to rank staff. Each person sees only their own shifts, and the roster flags anyone planned above 44 hours a week (normal hours under Singapore's Employment Act) [5].
- **Automation bias:** recommendations can be wrong, so they are explained, show the model's measured accuracy, and never place orders by themselves.
- **Honest reporting:** the ML model did not beat the baseline overall, and we report that rather than presenting only favourable numbers.

8.3 Reflection

What went well. Defining everything in CloudFormation meant the system could be rebuilt from nothing, which paid off when a design had to change. Writing measurable success criteria early lets us test each one, and evaluating the model against a strong baseline gives an honest answer instead of an assumed one.

What we would change. Initial scope included storage locations, per-batch expiry, and recipe sales, scaled back to deliver core ledger and forecasting systems. Scheduling was added once stable. Earlier AWS deployment would identify CloudFront limits sooner. Future additions include per-batch expiry, pack-size order rounding, holiday inputs, and secure in-house account setup.

9. References & Appendixes

APPENDIX I

Chosen service	Alternative	Why chosen, and the trade-off
EC2 Auto Scaling group (IaaS)	Elastic Beanstalk; ECS Fargate	Full control of server set-up (start-up script, hardening, log agent), with scaling and self-healing we configure and demonstrate directly. **Trade-off:** we patch the servers ourselves; Beanstalk would manage them but hide that configuration, and Fargate costs more for an always-on service.
API Gateway HTTP API	CloudFront; public load balancer	CloudFront was our first design, but cloudfront:CreateDistribution is denied in the Learner Lab. API Gateway gives an HTTPS address with an AWS certificate and no custom domain, throttles traffic (50 requests/s, bursts of 100), and lets the load balancer stay internal. **Trade-off:** no CDN edge caching, and WAF cannot see visitors' own IP addresses behind the VPC link.
Application Load Balancer (internal)	Pointing users at servers directly	Health checks every 15 s take a failed server out of service, traffic is spread over two Availability Zones, rolling updates avoid downtime, and WAF attaches to it. It only accepts API Gateway's VPC link. **Trade-off:** about $0.54 a day, our largest single cost.
AWS WAF	AWS Shield Advanced	Managed rule groups block injection, cross-site scripting and known bad inputs before they reach the servers. Shield Standard (network DDoS) is included free; Shield Advanced is priced for large enterprises.
Amazon RDS for PostgreSQL	Amazon Aurora; DynamoDB	Relational data with foreign keys and transactions: row locks keep concurrent stock updates correct, and per-café composite keys stop cross-café references in the database itself. Aurora's minimum cost is higher for no benefit at our size; DynamoDB would need these integrity rules rebuilt in code. **Trade-off:** single-AZ to save cost, so the database is a single point of failure (a Multi-AZ standby would double its cost).
Lambda + EventBridge	A scheduled job on the app servers	Runs once a night for a few minutes: cheaper than an always-on worker, never slows the web app, and runs exactly once even with two servers up. **Trade-off:** Lambda's size and time limits (timeout 5 minutes).
Amazon S3	Files on the servers; the database	Holds the app release, Lambda package and trained model; versioning keeps old ones 30 days for rollback.
Amazon Cognito	Our own user table with password hashes	Passwords never reach our servers; hashing, lockout and token signing are handled by AWS, and groups map directly to roles. **Trade-off:** accounts are managed in the AWS console, not in the app.
AWS Secrets Manager	SSM Parameter Store; environment files	Generates the database password so nobody types or sees it. **Trade-off:** about $0.25 a day with its VPC endpoint; Parameter Store is free but cannot generate or rotate passwords.
CloudWatch + SNS	AWS X-Ray; Amazon SES	Logs, metrics, dashboards and alarms in one service; alarms e-mail through SNS with no code. X-Ray traces requests across many services, which one API tier does not need; SES needs sender set-up and code.
AWS CloudFormation	Setting up in the console	The whole system rebuilds from six reviewed templates in about 1½ hours. **Trade-off:** slower to write, and Learner Lab restrictions (such as CloudFront) only appear at deploy time.

Table 1

Tests	Plan	Expected	Actual
1. Functional evidence/test-functional.md	As staff, record a delivery and a stock count. As manager, check the forecast, assign a shift and try an overlapping one. As staff, open My shifts.	Stock, Activity and Forecast show the entries. The shift appears in My shifts. The overlapping shift is refused.	**Pass**, 7 Oct 2026. The 2,000 ml milk delivery showed in Activity and Forecast. The overlap was refused and the shift showed for staff. AWS smoke test 10/10.
2. Security evidence/test-security.md	As staff, open the forecast and colleagues' shifts. Send anonymous and forged-token requests and a script probe to the WAF. Check that the database and load balancer aren't public	Staff refused (403). Anonymous and forged requests get 401. The WAF blocks the probe (403). The database and load balancer aren't reachable from the internet.	**Pass**, 7 Oct 2026. Returned 401 AUTH_REQUIRED and INVALID_TOKEN, and the WAF returned 403. The CSP, HTTPS and nosniff checks passed. The load balancer is internal and RDS isn't public.
3. Data and AI evidence/test-data-ai.md	Run the 19 analytics checks: same scoring days, no future data, deterministic training, reproducible metrics. Review the model chosen in the forecast-run logs.	All checks pass. The comparison is fair and reproducible.	**Pass**, 19/19, 1 Oct 2026. The latest run chose lgbm-v1: MASE 1.088 against the baseline's 1.126 (section 6.3).
4. Resilience evidence/test-resilience.md	With 2 servers running, send requests continuously for 15 minutes and terminate one server.	At most a few seconds of failed requests. Auto Scaling replaces the server.	**Partial**, 7 Oct 2026. 828 of 882 requests succeeded (93.88%), with a longest outage of 46 s. The server was replaced automatically, but scale-in cut capacity from 2 to 1 during recovery (section 6.2).

Table 2

References

[1] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," Int. J. Forecast., vol. 22, no. 4, pp. 679–688, 2006

[2] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, Long Beach, CA, USA, 2017, pp. 3146–3154.

[3] N. Kaushal, "Coffee Sales Dataset," Kaggle. Accessed: Oct. 9, 2026. [Online]. Available: https://www.kaggle.com/datasets/navjotkaushal/coffee-sales-dataset

[4] Creative Commons, "Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)." Accessed: Oct. 9, 2026. [Online]. Available: https://creativecommons.org/licenses/by-sa/4.0/

[5] Ministry of Manpower, Singapore, "Hours of work, overtime and rest days." Accessed: Oct. 9, 2026. [Online]. Available: https://www.mom.gov.sg/employment-practices/hours-of-work-overtime-and-rest-days